



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Semestre 1-2025
Técnicas Avanzadas de Programación



Asignación #4: Algoritmo de Kadane

Máxima suma 2D

Estudiante:
Erimar Reis CI 29.743.464

Caracas, 9 de Julio de 2025

Introducción

El problema de encontrar la submatriz con la suma máxima en una matriz 2D de números reales surge en múltiples contextos, desde análisis de imágenes hasta procesamiento de datos financieros. Una solución ingenua de fuerza bruta explora todas las posibles submatrices, lo que implica una complejidad de tiempo de $\mathcal{O}(n^6)$ o $\mathcal{O}(n^4)$ para algoritmos optimizados, esto es sumamente ineficiente para matrices de tamaño moderado. Para superar esta limitación, se adapta el algoritmo de Kadane, originalmente diseñado para arreglos 1D, a matrices bidimensionales, logrando una mejora significativa en eficiencia.

La idea central de Kadane 2D es reducir el problema 2D a una serie de problemas 1D. Esto se logra fijando pares de filas y colapsando las columnas entre esas filas en un arreglo unidimensional, sobre el cual se aplica Kadane 1D para encontrar la submatriz con suma máxima que se extiende verticalmente entre esas filas. Esta técnica reduce la complejidad a $\mathcal{O}(n^3)$, lo que la hace una opción a considerar para matrices de tamaño considerable.

Fundamento del Algoritmo

El Algoritmo de Kadane 2D se basa en los siguientes pasos:

1. Se recorren todos los pares posibles de filas superiores e inferiores.
2. Para cada par de filas, se construye un arreglo temporal que representa la suma acumulada de cada columna entre esas dos filas.
3. Sobre este arreglo temporal, se aplica Kadane 1D para encontrar el subarreglo con la suma máxima.
4. Se actualiza el resultado global si la suma encontrada es mayor que la máxima registrada hasta el momento.

Este enfoque aprovecha la complejidad de Kadane 1D $\mathcal{O}(n)$ y la combina con una iteración sobre pares de fila (con complejidad $\mathcal{O}(n^2)$), resultando en una complejidad total de $\mathcal{O}(n^3)$, mejorando la eficiencia de la implementación ingenua.

Metodología Experimental

Para evaluar la eficiencia del algoritmo de Kadane 2D ($\mathcal{O}(n^3)$) frente a la versión fuerza bruta optimizada ($\mathcal{O}(n^4)$), se diseñó un experimento controlado que mide el tiempo de ejecución promedio en función del tamaño de la matriz. Se consideraron valores de n en el rango $[5, 100]$, con incrementos de 5 unidades. Para cada valor de n , se generaron 20 matrices distintas de tamaño $n \times n$ con valores reales aleatorios en el intervalo $[-100, 100]$. La generación de cada matriz i -ésima (con $i = 1..20$) se realizó con una probabilidad de $\frac{i}{20}$ de que cada celda contuviera un valor positivo, lo que permitió simular distintos niveles de densidad de valores positivos en los datos. `float probpositivo = (float)(iter)/ 20.0f;`

La función de generación de matrices utilizó una distribución uniforme tanto para los valores como para la probabilidad, y se inicializó con una semilla específica para garantizar reproducibilidad. Las 20 semillas utilizadas fueron las mismas para ambas implementaciones, asegurando que cada algoritmo procesara exactamente las mismas matrices en cada iteración. Ambos algoritmos fueron evaluados utilizando exactamente los mismos tamaños de n y las mismas matrices generadas. Para cada combinación de n y matriz, se midió el tiempo de ejecución en microsegundos. Posteriormente, se calculó el tiempo promedio de ejecución para cada valor de n a partir de las 20 ejecuciones correspondientes.

Este enfoque permitió comparar de forma justa y directa el rendimiento de ambas soluciones bajo condiciones idénticas, aislando el impacto del algoritmo sobre el tiempo de ejecución.

Análisis de resultados

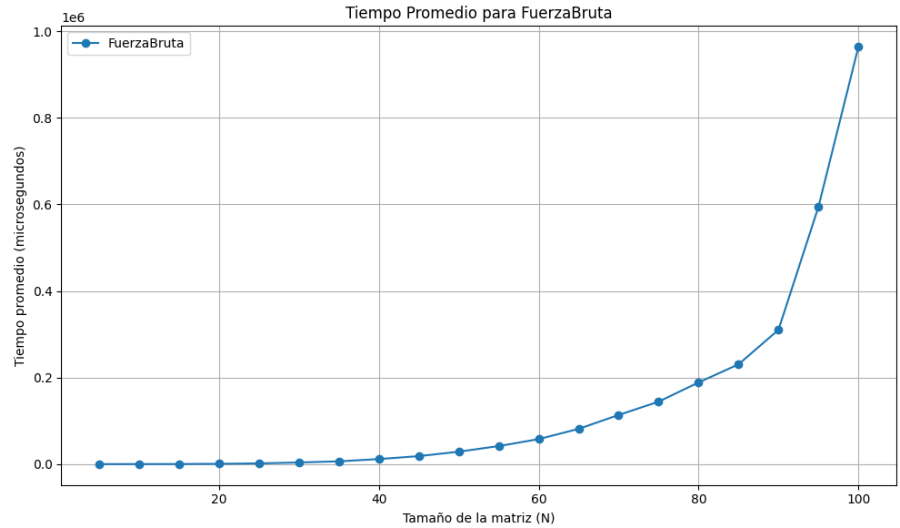
Con el objetivo de comparar el rendimiento entre la versión fuerza bruta y el algoritmo de Kadane 2D, se analizaron los tiempos de ejecución obtenidos para distintos tamaños de matriz. Con la intención de facilitar la ejecución sistemática de las pruebas, se desarrolló un script en Python que compila y ejecuta ambos archivos .cpp, este script automatiza la recolección de los tiempos de ejecución y genera gráficas comparativas que nos permiten ver la evolución del tiempo promedio en función del tamaño de la matriz.

A continuación, se presentan los resultados obtenidos, los cuales permiten visualizar claramente la diferencia de eficiencia entre ambas soluciones conforme crece el tamaño de entrada.

Rendimiento de Fuerza Bruta

El algoritmo de fuerza bruta para encontrar la submatriz de suma máxima tiene una complejidad temporal de $\mathcal{O}(n^4)$. Esto se debe a que itera sobre todas las posibles submatrices, lo cual implica cuatro bucles anidados: dos para definir las filas superior e inferior, y dos para definir las columnas izquierda y derecha. Dentro de estos bucles, el cálculo de la suma de la submatriz se optimiza usando una matriz de suma de prefijos, lo que reduce el cálculo de la suma de cada submatriz a $\mathcal{O}(1)$. Sin esta optimización, el cálculo de cada submatriz tomaría $\mathcal{O}(n^2)$, resultando en una complejidad total de $\mathcal{O}(n^6)$. Una vez realizado el experimento se obtuvieron los siguientes resultados:

Tamaño n	Tiempo promedio (μs)
5	10.65
10	108.8
15	242.7
20	667.6
25	1676.3
30	3751.2
35	6400.1
40	11666.2
45	18469.1
50	28713.6
55	41693.6
60	57757
65	81514.1
70	113513
75	144267
80	188693
85	230552
90	310318
95	594341
100	964149

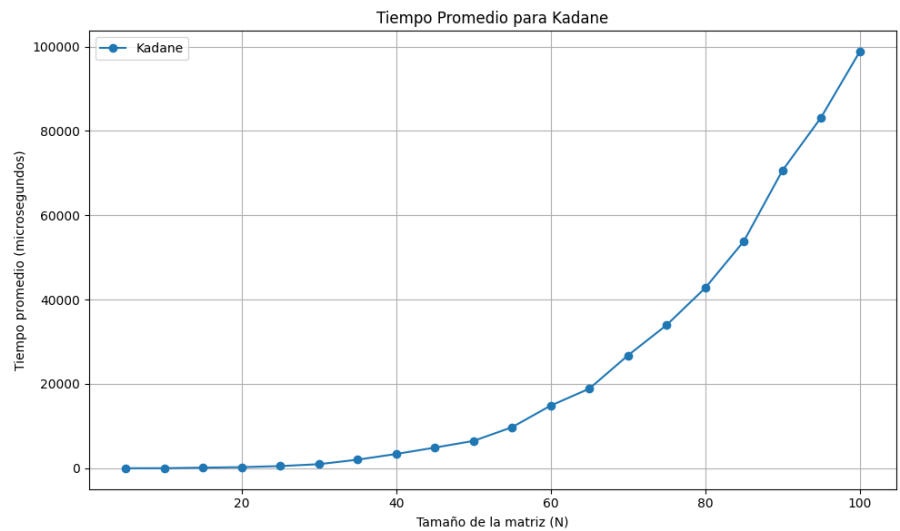


Como se puede observar en la tabla y en el gráfico, el tiempo promedio de ejecución aumenta drásticamente a medida que el tamaño de la matriz N crece. Para $N = 5$, el tiempo es de $10.65 \mu s$, mientras que para $N = 100$, asciende a $964149 \mu s$ (aproximadamente 0.96 segundos), este crecimiento exponencial es característico de los algoritmos con complejidad polinomial de alto grado. La curva en el gráfico es muy pronunciada, lo que confirma el impacto significativo del aumento de N en el tiempo de ejecución.

Rendimiento de Kadane 2D

El algoritmo de Kadane 2D para la submatriz de suma máxima tiene una complejidad temporal de $\mathcal{O}(n^3)$. Este algoritmo funciona comprimiendo el problema 2D en múltiples problemas 1D, los cuales son resueltos utilizando el algoritmo de Kadane 1D, que tiene una complejidad de $\mathcal{O}(n)$. El proceso implica iterar sobre todas las posibles filas superiores e inferiores ($\mathcal{O}(n^2)$ pares de filas), y para cada par, se "comprimen" las filas intermedias en un arreglo 1D, sobre el cual se aplica el algoritmo Kadane 1D. Una vez realizado el experimento se obtuvieron los siguientes resultados:

Tamaño n	Tiempo promedio (μs)
5	14.4
10	31.55
15	158.75
20	276.35
25	526.85
30	972.7
35	2044.45
40	3391.35
45	4907.75
50	6483.8
55	9739.05
60	14861.7
65	18877.2
70	26758.6
75	33950.4
80	42750.5
85	53824.1
90	70592.9
95	83165.5
100	98764.3

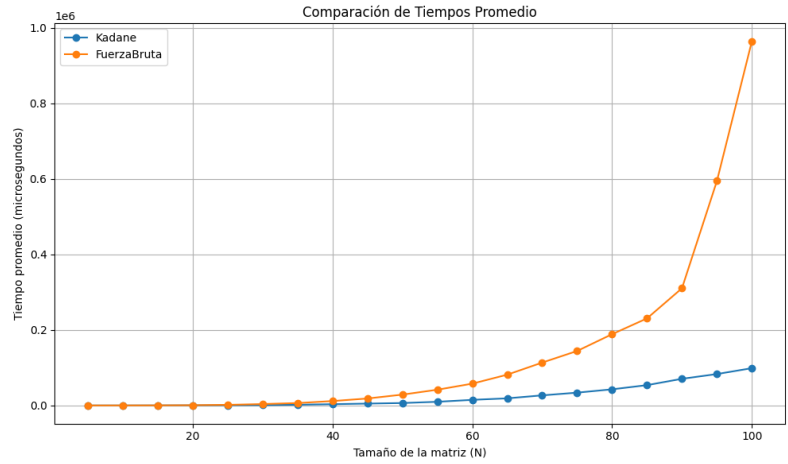


El gráfico y la tabla nos muestran que el tiempo de ejecución también aumenta con N , pero de manera mucho menos abrupta que el algoritmo de fuerza bruta. Por ejemplo, para $N = 5$, el tiempo es de $14.4 \mu s$, y para $N = 100$, el tiempo es de $98764.3 \mu s$ (aproximadamente 0.098 segundos), aunque sigue siendo un crecimiento significativo, es más eficiente que la fuerza bruta para valores grandes de N . La curva del gráfico de Kadane 2D es más suave, reflejando su menor complejidad.

Comparación de Kadane 2D ($\mathcal{O}(n^3)$) y Fuerza Bruta ($\mathcal{O}(n^4)$)

A continuación se muestra a modo de resumen los resultados obtenidos, esto para notar la diferencia de rendimiento entre ambos algoritmos:

Tamaño n	Kadane 2D (μs)	Fuerza Bruta (μs)
5	14.4	10.65
10	31.55	108.8
15	158.75	242.7
20	276.35	667.6
25	526.85	1676.3
30	972.7	3751.2
35	2044.45	6400.1
40	3391.35	11666.2
45	4907.75	18469.1
50	6483.8	28713.6
55	9739.05	41693.6
60	14861.7	57757
65	18877.2	81514.1
70	26758.6	113513
75	33950.4	144267
80	42750.5	188693
85	53824.1	230552
90	70592.9	310318
95	83165.5	594341
100	98764.3	964149



A partir del análisis de los gráficos de rendimiento con distintos tamaños de matriz, se observa que la principal diferencia entre los algoritmos de Fuerza Bruta y Kadane 2D radica en su complejidad: $\mathcal{O}(n^4)$ frente a $\mathcal{O}(n^3)$, respectivamente. Esta diferencia se traduce en una ventaja creciente para Kadane 2D a medida que N aumenta. Aunque para tamaños pequeños (como $N = 5$) la Fuerza Bruta es un poco más rápida (10.65 μs vs. 14.4 μs), esta ventaja desaparece rápidamente. El breve retardo de Kadane 2D en entradas pequeñas puede explicarse por factores como llamadas a funciones y operaciones adicionales, pero su estructura más eficiente se hace notoria conforme crece la escala de datos.

Para valores grandes de N , Kadane 2D demuestra una superioridad clara y constante. Por ejemplo, para $N = 20$ ya es más rápido (276.35 μs vs. 667.6 μs), y para $N = 100$ la diferencia es sumamente grande (98,764.3 μs vs. 964,149 μs). Esta tendencia confirma las predicciones teóricas: la curva de crecimiento de Fuerza Bruta es mucho más pronunciada, mientras que Kadane 2D escala de forma más controlada.

Conclusiones

De este estudio podemos concluir que los resultados prácticos confirman las complejidades teóricas de ambos algoritmos: la curva de crecimiento de Fuerza Bruta es mucho más pronunciada que la de Kadane 2D. Esto se traduce en una ventaja clara para Kadane 2D en matrices grandes, donde su eficiencia lo convierte en la opción más adecuada.

Aunque la Fuerza Bruta fue optimizada con sumas de prefijos para evitar una complejidad aún mayor, su rendimiento sigue siendo inferior en escalas de datos grandes. Por tanto, Kadane 2D no solo es más eficiente teóricamente, sino también en la práctica, especialmente cuando se trabaja con entradas de tamaño considerable.

Referencias

- [Kadane's Algorithm for 2D Matrices - Medium](#)
- [Kadane's Algorithm \(Dynamic Programming\) How and Why Does it Work? - Medium](#)
- [Maximum sum rectangle in a 2D matrix - GeeksForGeeks](#)
- [Kadane Algorithm - Leetcode The Hardway](#)
- [DeepSeek - AI Chat](#)